nSense: Limits of Bias

Harrison Dean
University of Victoria
harrisondean@uvic.ca

Carson Irvine
University of Victoria
carsonirvine@uvic.ca

Xavier Roepcke
University of Victoria
xavierroepcke@uvic.ca

Matthew Dunfee
University of Victoria
m1dunfee@uvic.ca

ABSTRACT

Bias tests for embeddings, such as WEAT, rely on hand-picked word lists. These word lists must separate words whose social group is part of their meaning (e.g., *queen* for female) from words tied to a group only by stereotype (e.g., *nurse* for female). To automate this separation, we propose nSense, a model that classifies words in sentences as *innate*, *derived*, or *neutral*. nSense uses the word's contextual BERT embedding and a random forest trained on StereoSet, and is evaluated on words it has never seen. The code base, along with instructions, can be found at:
https://github.com/Hammerhead-lab/nSense.

KEYWORDS

Embedding bias, AI, vector space, innate bias, derived bias

1 PROBLEM DEFINITION

Our goal is to identify, for a word in a sentence about a social group, whether the group is part of the word's meaning, the word is tied to the group by stereotype alone, or neither. Bias tests for embeddings depend on word lists that make this distinction, yet prior work either leaves stereotyped and neutral words mixed or labels them by hand [1]. Here, we provide a formal statement of the problem after which we propose our approach in detail in the next section.

We view this problem as an instance of multi-class classification. Let $\mathbb{G}$ be a set of social groups on three axes: gender, race/nationality, and religion. An observation $o = (s, i) \in \mathbb{O}$ pairs a sentence s about a target group $g \in \mathbb{G}$ with a character span i that selects a word w in s. The class $y(o) \in \mathbb{Y}$ is defined relative to g:

$$y(o) = \begin{cases} \text{innate} & \text{if } g \text{ is part of the meaning of } w, \\ \text{derived} & \text{if } w \text{ is tied to } g \text{ only by stereotype,} \\ \text{neutral} & \text{otherwise} \end{cases} \quad (1)$$

The goal of bias detection is to learn $f : \mathbb{O} \to \mathbb{Y}$ such that $f(o) = y(o)$, including for words not seen in training.

2 PROPOSED APPROACH

Our proposed bias detection method seeks to classify word spans as innate, derived, or neutral using labels constructed from StereoSet[5]. Each sentence is passed through pretrained BERT to produce a 768-dimensional vector representation of the selected word span. A random forest with 500 decision trees then learns to predict the word span's class from this vector. The classifier is evaluated using five-fold grouped cross-validation, keeping the training and testing word groups separate, and then its performance is compared with four baselines using macro-F1.

2.1 Observations (StereoSet)

We construct observations from StereoSet's intrasentence validation data containing single-sentence examples focusing on gender, race/nationality, and religion. Each StereoSet item contains a target group term and sentence variants labeled either stereotype, anti-stereotype, or unrelated. An observation o consists of a sentence s and a selected word span i represented as o = (s, i), with each observation being paired with a reference label. We assign innate to the target term in the stereotype sentence, derived to the word filling its blank, and neutral to the fill word in the unrelated sentence. We excluded anti-stereotype sentences and profession items from our observations, as they do not fit the three classes. As well, we set aside derived or neutral spans that match a retained group term. This process resulted in 3,857 labeled observations, which are the input for the next stage, BERT encoding.

2.2 Encoding (BERT)

To establish credibility, the model, like the dataset, was chosen to mirror the research in the topic literature. BERT-base was among the models evaluated in the original StereoSet paper [5], and bert-base-uncased is the BERT model used in bias-bench [4]. The reason uncased is used here is because the boundaries include innate words, which include proper nouns, and we want to exclude case-sensitive details from the vectors to avoid the random forest learning to treat those groups differently on that feature alone. BERT configuration includes being frozen with no fine-tuning, using the final hidden layer, mean pooling over subword pieces, and one 768-dimensional vector per target word in its encoded sentence. These vectors are the forest's input.

2.3 Random Forest Classifier

Given the 768-dimensional vector representation $x_o \in \mathbb{R}^{768}$ extracted for each labeled word, we train a random forest model [2] $c : \mathbb{R}^{768} \to \{\textit{innate}, \textit{derived}, \textit{neutral}\}$ to classify each word. A random forest works by building an ensemble of decision trees. In our setup, $T = 500$ individual trees—where each tree is trained on a random bootstrap sample of the rows in the training set. At every decision point (or split) in a tree, the algorithm randomly picks a small subset of features (here, $\lfloor\sqrt{768}\rfloor = 27$ dimensions) and selects the threshold that best splits the data into distinct classes. The tree keeps splitting the data into narrower branches until every final leaf node contains only a single class.

When making a prediction for a word vector $\mathbf{x}_o$, each tree calculates a probability distribution across the three classes, and the

Algorithm 1 Classifying words as innate, derived, or neutral

Inputs:
$\mathcal{I}$, StereoSet items, each with a target word τ,
a stereotype sentence s^+, and an unrelated sentence s^-;
$\mathcal{B}$, pretrained BERT encoder;
K, number of cross-validation folds;
Output:
c^*, trained random forest;
$\overline{F1}$, mean macro-F1 over held-out folds;
Initialization:
$\mathcal{R} = \emptyset$, set of labeled words;
set_seed(σ);

procedure nSense($\mathcal{I}, \mathcal{B}, K, \sigma$)
  for all items $(\tau, s^+, s^-) \in \mathcal{I}$ **do**
    $w^+ \leftarrow$ fill word of s^+
    $w^- \leftarrow$ fill word of s^-
    $\mathcal{R} \leftarrow \mathcal{R} \cup \{(s^+, \tau, \textit{innate}),\ (s^+, w^+, \textit{derived}),\ (s^-, w^-, \textit{neutral})\}$
  for all $(s, w, y) \in \mathcal{R}$ **do**
    $\mathbf{x} \leftarrow$ mean of $\mathcal{B}(s)$ token vectors of w
  split $\mathcal{R}$ into K folds so that each word group is in one fold only
  for each fold k **do**
    train random forest on the other folds; predict fold k
    $F1_k \leftarrow$ macro-F1 on fold k
  $c^* \leftarrow$ random forest trained on all of $\mathcal{R}$
  return c^*, $\overline{F1} = \text{mean}_k\ F1_k$

forest averages these probabilities together:

$$c(x_o) = \arg\max_y \frac{1}{T} \sum_{t=1}^{T} p_t(y \mid x_o)$$

The final predicted label is simply the class with the highest average probability p_t across all 500 trees. Aside from setting $T = 500$ and locking the random seed to 503, we rely on the standard scikit-learn default settings [6]. We deliberately skip hyperparameter tuning because generic random forests consistently deliver strong performances [7].

To evaluate how well our model generalizes to unseen words, we run 5-fold cross-validation using GroupKFold [8]. We group the rows by their base word (using the word's lemma, or the normalized target word for innate entries). This strict grouping ensures that no single word ever appears in both the training set and the test set within the same fold, preventing the model from simply memorizing specific words.

We measure performance using macro-averaged F1 score [9], which treats all three target classes equally regardless of how many examples each class has. We report the mean and standard deviation of the macro-F1 score across all five test folds. In addition, we measure feature importance by running permutation importance on each held-out test fold [2, 3]. To judge our model's performance fairly, we evaluate four baseline models: a single cosine direction tree, a part-of-speech tag classifier, a word frequency classifier, and a context-only forest. We then use the exact same cross-validation folds and scoring metrics.

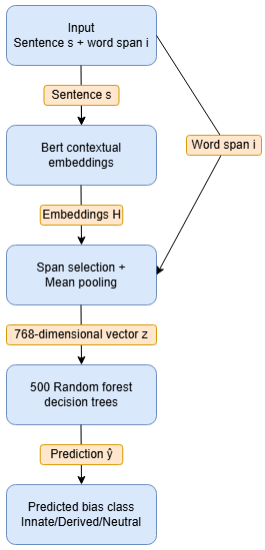


Figure 1: The nSense classification pipeline.

REFERENCES

[1] Tolga Bolukbasi, Kai-Wei Chang, James Y Zou, Venkatesh Saligrama, and Adam T Kalai. 2016. Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings. In *Advances in Neural Information Processing Systems (NeurIPS 2016)*, Vol. 29. 4349–4357. https://arxiv.org/abs/1607.06520

[2] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32.

[3] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. 2019. All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously. *Journal of Machine Learning Research* 20, 177 (2019), 1–81.

[4] Nicholas Meade, Elinor Poole-Dayan, and Siva Reddy. 2022. An Empirical Survey of the Effectiveness of Debiasing Techniques for Pre-trained Language Models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.). Association for Computational Linguistics, Dublin, Ireland, 1878–1898. doi:10.18653/v1/2022.acl-long.132

[5] Moin Nadeem, Anna Bethke, and Siva Reddy. 2020. StereoSet: Measuring stereotypical bias in pretrained language models. arXiv:2004.09456 [cs.CL]

[6] Fabian Pedregosa et al. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.

[7] Philipp Probst, Marvin N. Wright, and Anne-Laure Boulesteix. 2019. Hyperparameters and Tuning Strategies for Random Forest. *WIREs Data Mining and Knowledge Discovery* 9, 3 (2019), e1301.

[8] David R. Roberts et al. 2017. Cross-validation Strategies for Data with Temporal, Spatial, Hierarchical, or Phylogenetic Structure. *Ecography* 40, 8 (2017), 913–929.

[9] Marina Sokolova and Guy Lapalme. 2009. A Systematic Analysis of Performance Measures for Classification Tasks. *Information Processing & Management* 45, 4 (2009), 427–437.